

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Ярославский государственный технический университет»
Кафедра «Информационные системы и технологии»

Отчет защищен
с оценкой _____
Преподаватель
_____ Д.В. Малышев
«__» _____ 2025

АРХИТЕКТУРА И ПАТЕРНЫ WEB- ПРОГРАММИРОВАНИЯ

Отчет о лабораторной работе №2
ЯГТУ 09.03.02 – 001 ЛР

Отчет выполнил
студент гр. ЦИС-37
Нахтигал Д.С.
«_» _____ 2025

Цель работы: изучить и реализовать различные структурные паттерны проектирования.

Задание: реализовать с использованием структурных паттернов web приложение

Вариант 1:

Реализовать API-шлюз , который преобразует данные одного формата в другой, который ожидает клиентское веб-приложение (Адаптер) .

Пример: Клиент ожидает объект с полем price, а ему приходит поле cost.

Описание процесса работы:

Прежде всего надо установить Node.js

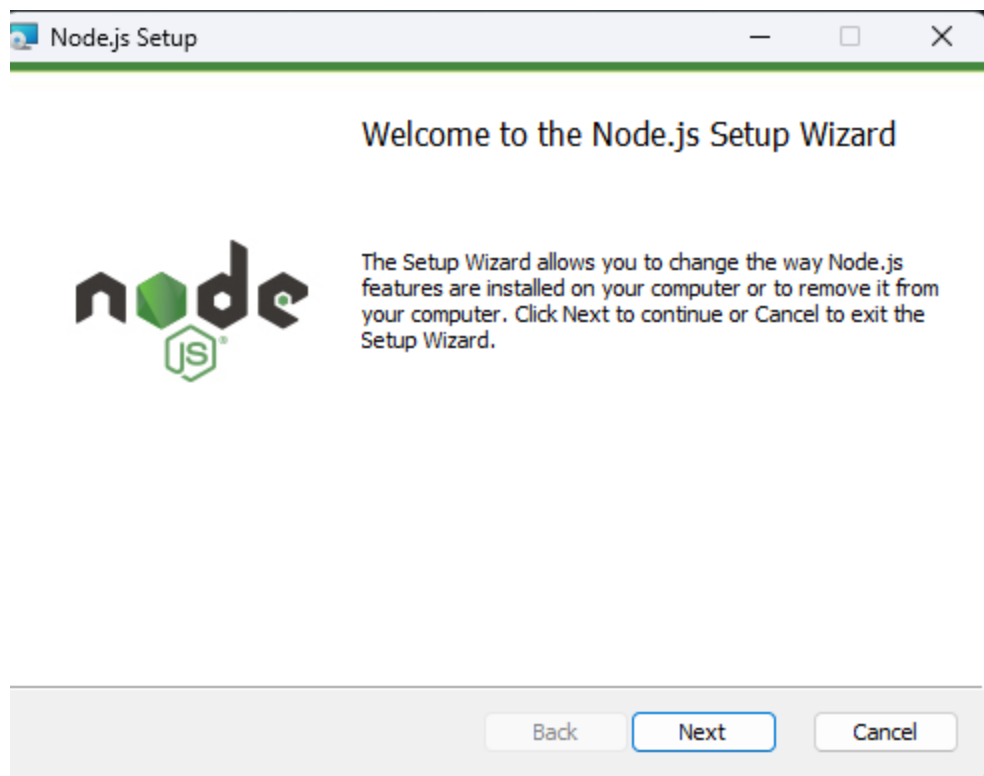


Рисунок 1 – установщик Node.js

Далее создаётся и инициализируем проект

```

PS C:\Users\zrhen> node -v
v22.14.0
PS C:\Users\zrhen> mkdir api-gateway-adapter

Каталог: C:\Users\zrhen

Mode                LastWriteTime         Length Name
----                -
d-----         18.03.2025         8:43             api-gateway-adapter

PS C:\Users\zrhen> cd api-gateway-adapter

```

Рисунок 2 – создание папки проекта

```

PS C:\Users\zrhen\api-gateway-adapter> npm init -y
Wrote to C:\Users\zrhen\api-gateway-adapter\package.json:

{
  "name": "api-gateway-adapter",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

```

Рисунок 3 – инициализация

Был поставлен Фреймворк для сервера и позволено клиенту взаимодействовать с ним

```

PS C:\Users\zrhen\api-gateway-adapter> npm install express cors
added 71 packages, and audited 72 packages in 2s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\zrhen\api-gateway-adapter> node backend.js

```

Рисунок 4 – настройка node.js

После был прописан бэк, а точнее преобразование и принятие значение отправленного пользователем.

```

const express = require("express"); //облегчает создание сервера
const cors = require("cors"); //разрешает запросы с других доменов для фронта
const bodyParser = require("body-parser"); //позволяет читать JSON-данные из POST-запросов

```

```

//Создание и наастройkaa сервака
const app = express();
app.use(cors());
app.use(bodyParser.json());

let externalData = { cost: 915 }; // Начальное значение

// Отправляем преобразованные данные через паттерн "АААДДАААПТЕР"
app.get("/api/price", (req, res) => {
  res.json({ price: externalData.cost });
});

// Принимаем новое значение cost от клиента
app.post("/api/price", (req, res) => {
  const newCost = req.body.cost;
  if (typeof newCost === "number" && newCost > 0) {
    externalData.cost = newCost; // Обновляем значение
    res.json({ success: true, price: externalData.cost });
  } else {
    res.status(400).json({ success: false, message: "значеньеце не то" });
  }
});

app.listen(3000, () => console.log("сервачёк на http://localhost:3000"));

```

Код 1 – backend.js

После написания бэка, потребуется фронт, через который пользователь сможет вводить значение и передавать его на сервер, так что был создан файл html

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>адаптер</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Данные с API-шлюза</h1>
  <div id="data-container">Загрузка...</div>

  <h2>Изменить цену</h2>
  <input type="number" id="price-input" placeholder="Введите новое значение">

```

```
<button onclick="updatePrice()">Обновить</button>

<script src="script.js"></script>
</body>
</html>
```

Код 2 – index.html

Чуть выровняем css-ом

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 20px;
}
#data-container {
  font-size: 20px;
  margin-top: 20px;
  padding: 10px;
  border: 1px solid #000;
  display: inline-block;
}
```

Код 3 – styles.css

А теперь сделаем скриптовую часть фронта, для начала в файле script.js пропишем взятие данных с сервера для отображения изначального значения на сервере

```
//данные с сервера
async function fetchData() {
  try {
    const response = await fetch("http://localhost:3000/api/price");
    const data = await response.json();
    document.getElementById("data-container").innerText = `Цена: ${data.price}`;
  } catch (error) {
    document.getElementById("data-container").innerText = "Ошибка загрузки данных";
  }
}
```

Код 4 – данные с сервера

Далее после того как пользователь введет значение на сайте проверяем на корректность введенное значение, а после отправляем его post запросом на сервер, в конце вызываем первую функцию, для обновления данных на странице

```
//передача данных на сервер
async function updatePrice() {
    const newPrice = document.getElementById("price-input").value;
    if (!newPrice || isNaN(newPrice) || newPrice <= 0) {
        alert("Введите корректную цену!");
        return;
    }

    try {
        const response = await fetch("http://localhost:3000/api/price", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ cost: Number(newPrice) })
        });

        if (response.ok) {
            fetchData(); // Обновляем данные на экране
        } else {
            alert("Ошибка обновления цены");
        }
    } catch (error) {
        alert("Ошибка соединения с сервером");
    }
}

fetchData();
```

Код 5 – script.js

Отображение работы:

Страница index до ввода значения

Данные с API-шлюза

Цена: 915

Изменить цену

Введите новое значение

Обновить

Рисунок 5 – index.html

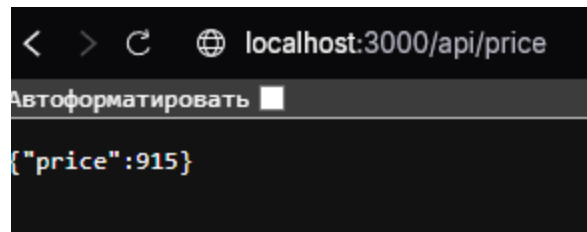


Рисунок 6 – значение на сервере

Данные с API-шлюза

Цена: 123435678890

Изменить цену

123435678890

Обновить

Рисунок 7 – пользователь обновил значение

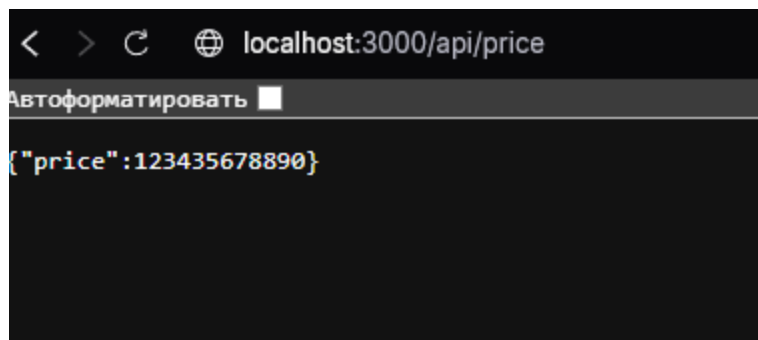


Рисунок 8 – измененное значение на сервере

Вывод: Был реализован API-шлюз , который преобразует данные одного формата в другой, который ожидает клиентское веб-приложение (Адаптер)